

Foundations of R Chapter 7

Robert Gulotty

11/10/2018

This chapter returns to the topic of Chapter 2, statistical inference, by walking through an analysis of the World Series. The programming goal in this chapter is to perform many of the same steps, modeling, simulation and analysis, but in this case to use the vectorized tools from Chapter 6, especially the very powerful map functions from the purrr package.

BASEBALL!

The World Series is a best of 7 contest, whoever wins four wins the series wins, and they stop playing. Our question is how long the series will last. If you are unfamiliar with baseball, I suggest watching the documentary **A League of Their Own**. In the following, we will assume that the probability of winning is just .5, and then we will explore the fit of this assumption to the data.

```
p <- 0.5
```

Review of Combinatorics

Our analytical strategy is to enumerate cases and then generalize. First we will consider the probability of each outcome, then we will develop a function to enter into our simulation..

The probability of the world series ending in exactly four games is the sum of a win, a win, a win and a win or a loss, a loss, a loss, and a loss. That is,

```
winin4 <- p * p * p * p + (1 - p) * (1 - p) * (1 - p) * (1 - p)
winin4
```

```
## [1] 0.125
```

Because the two teams are symmetric, we could have just multiplied by two:

```
2 * p * p * p * p
```

```
## [1] 0.125
```

Now if it is going to end in 5 games, the series can work several ways, without loss of generality, the home team could lose the first, the second, the third or the fourth games, but they have to win the final game. These are four possibilities:

```
winin5 <- 2 * 4 * p * p * p * (1 - p) * p
winin5
```

```
## [1] 0.25
```

If the series lasts for 6 games, they could lose the first two, the first and the second to last, etc. This is a combinatoric problem, and can be calculated with the choose function. Take the number of things in the set that you are ordering, call that n, and then the number of distinct elements as k. Then `choose(n, k) = n!/(k!(n-k)!)` is the number of k-combinations.

So for the previous example, we had four games before the last one that could be in any order, so n=4, and we had one element that was distinct, k=1.

```
winin5 <- 2 * choose(4, 1) * p * p * p * (1 - p) * p
winin5
```

```
## [1] 0.25
```

We can extend this logic with 5 games and two losses

```
winin6 <- 2 * choose(5, 2) * p * p * p * (1 - p) * (1 - p) * p
winin6
```

```
## [1] 0.3125
```

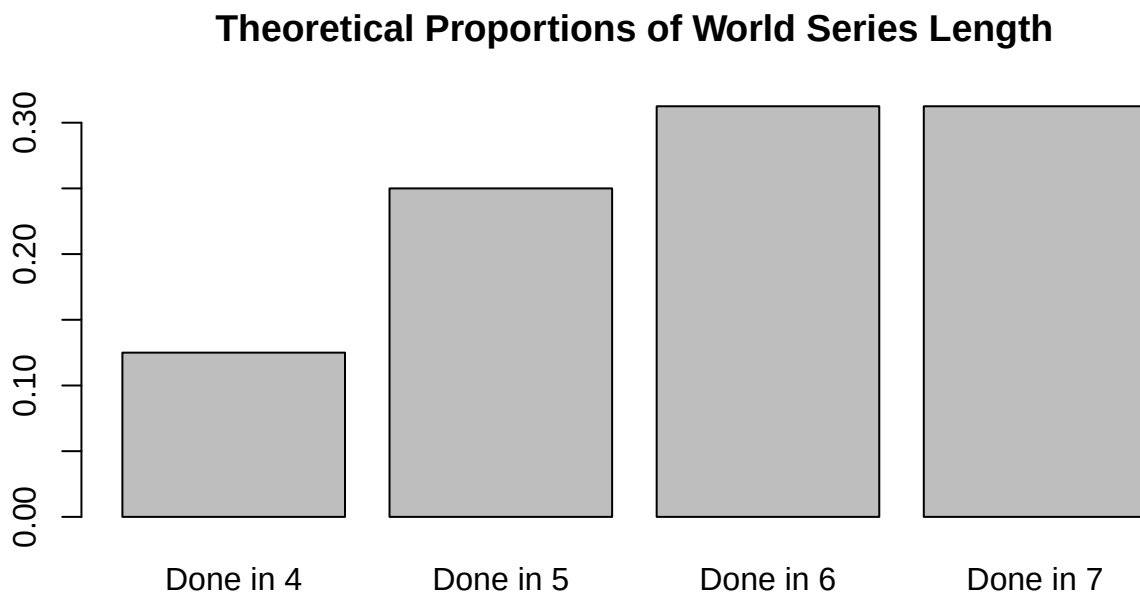
We can extend this logic with 6 games and three losses

```
winin7 <- 2 * choose(6, 3) * p * p * p * (1 - p) * (1 - p) * (1 - p) * p
winin7
```

```
## [1] 0.3125
```

Given these results, the following code displays the theoretical results graphically:

```
tprops <- c(winin4, winin5, winin6, winin7)
names(tprops) <- c("Done in 4", "Done in 5", "Done in 6", "Done in 7")
barplot(tprops, main = "Theoretical Proportions of World Series Length")
```



Isn't it surprising that the ex ante probability of the series being over in six games is equal to it being done in 7 games? A quick look ahead to conditional probability will give you a sense for why. Suppose that you just finished Game 5, and it is not over. There are many ways you could have gotten to this point, but we will ignore that for now. One of the two teams is in a position to win it all if they win Game 6. If that team loses Game 6, then you go to Game 7. The chance of that happening is just the chance that a team wins a game, which is .5. So with equal probability, the games go to either 6 or 7 games.

Demonstrating the form of the binomial distribution.

The preceding logic is related to one of the most useful distributions in probability, the binomial distribution. We will learn about random variables and their distributions soon, but the binomial distribution describes the probability of k successes in n independent trials of a binary experiment where each experiment has the same probability of success p .

$$f(k; n, p) = P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

The simplest case is the single trial experiment, like a coin toss, where $n=1$.

$$f(k; n, p) = \begin{cases} P(X = 1) = \binom{1}{1} p^1 (1 - p)^{1-1} = p \\ P(X = 0) = \binom{1}{0} p^0 (1 - p)^{1-0} = 1 - p \end{cases}$$

The more complicated situation arises whenever you count things that arise from a fixed constant process, like the number of games that a world series lasts.

I suggest you work through the following equations to reassure yourself of their similarity.

```
endin4 <- 2 * choose(3, 3) * p * p * p * p
endin4binomial <- 2 * dbinom(x = 3, size = 3, prob = p) * p

endin5 <- 2 * choose(4, 3) * p * p * p * (1 - p) * p
endin5binomial <- 2 * dbinom(x = 3, size = 4, prob = p) * p

endin6 <- 2 * choose(5, 2) * p * p * p * (1 - p) * (1 - p) * p
endin6binomial <- 2 * dbinom(x = 3, size = 5, prob = p) * p

endin7 <- 2 * choose(6, 3) * p * p * p * (1 - p) * (1 - p) * (1 - p) * p
endin7binomial <- 2 * dbinom(x = 3, size = 6, prob = p) * p
```

Evaluation through simulation methods

Humans are famously bad at thinking about probability and how changes in our assumptions affect outcomes. One way to overcome these limitations is to simulate data. Here we will simulate the data generating process for the baseball games, and then run a simulation of these games to check our distributional assumptions. We proceed with the principles of programming, breaking up our problem into sub-parts.

First we write a function, `worldseriesgame` which takes as input our probability of winning and returns a single sample, a single up or down win or loss for the home team.

```
worldseriesgame <- function(pwin) {  
  outcomeofgame <- sample(c(0, 1), size = 1, prob = c(pwin, 1 - pwin))  
  return(outcomeofgame)  
}
```

Then we embed that function into another function, `worldseries` which takes as input our probability of winning and the total wins needed to end the series and returns the length of the series.

```
worldseries <- function(totalwinsneeded, pwin) {  
  maxgames <- 25  
  gameswonbyA <- replicate(worldseriesgame(pwin), n = maxgames)  
  
  totalAwins <- cumsum(gameswonbyA)  
  totalBwins <- cumsum(1 - gameswonbyA)  
  
  EndbyA <- min(c(which(totalAwins == totalwinsneeded), maxgames))  
  EndbyB <- min(c(which(totalBwins == totalwinsneeded), maxgames))  
  
  Serieslength <- min(EndbyA, EndbyB)  
  return(Serieslength)  
}
```

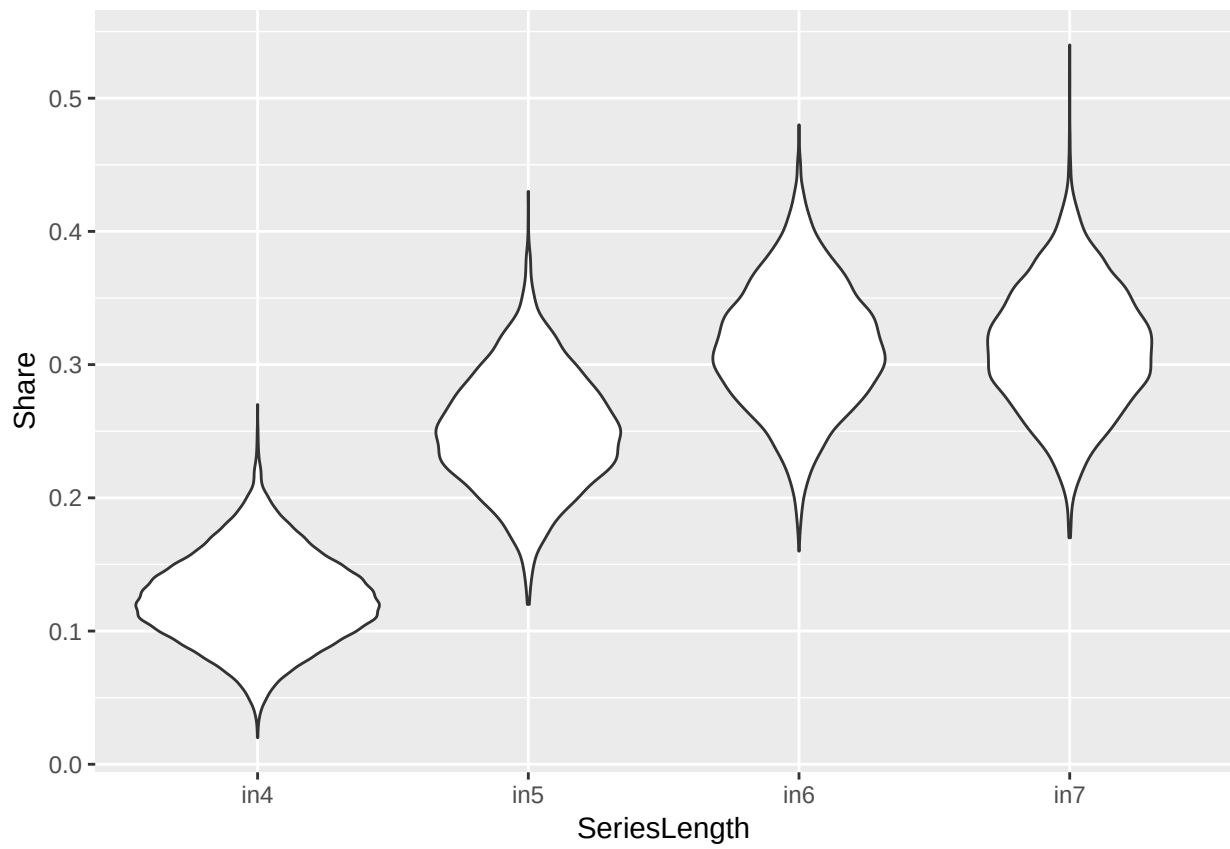
Now that we have a `worldseries` function, we can simulate one run of 100 years:

```
sim100 <- replicate(worldseries(4, 0.5), n = 100)  
table(sim100)/sum(table(sim100))
```

```
## sim100  
##      4      5      6      7  
## 0.14 0.18 0.36 0.32
```

What if we ran that 100 years for 5000 worlds?

```
worldhistory <- function(maxwins) {  
  sim100 <- replicate(worldseries(maxwins, 0.5), n = 100)  
  table(sim100)/sum(table(sim100))  
}  
  
maxwins <- 4  
athousandworlds <- replicate(worldhistory(maxwins), n = 5000)  
  
dfthousand <- data.frame(t(athousandworlds))  
names(dfthousand) <- paste0("in", maxwins:(2 * maxwins - 1))  
  
toplot <- gather(dfthousand)  
  
ggplot(toplot, aes(key, value)) + geom_violin() + labs(x = "SeriesLength", y = "Share")
```



Comparing theory to data

Ok, now we have a model for the world series. We should fit it to the data.

Read in data from Wikipedia.

```
WSeries <- read_csv("~/Google Drive/IPS2014/Lecture3/WSeries.csv")
```

```
## Parsed with column specification:
## cols(
##   Year = col_integer(),
##   Winners = col_character(),
##   Wmanager = col_character(),
##   Wleague = col_character(),
##   Games = col_character(),
##   Losers = col_character(),
##   Lmanager = col_character(),
##   Lleague = col_character()
## )
```

```
# Drop canceled World Series.
```

```
WSeriesin4 <- WSeries %>% filter(!is.na(Games))
```

```
## Warning: package 'bindrcpp' was built under R version 3.4.4
```

To clean this data, we need to turn the 4xY nomenclature to 4+Y length of game. Here we use the `str_extract_all` function, which takes as input a regular expression for digit and returns a list. The first map function then applies `as.numeric` to that list and the second map function applies the sum command.

```
WSeriesin4 <- WSeriesin4 %>% mutate(length = map_dbl(map(str_extract_all(Games, "\\d"),
  as.numeric), sum))
```

At this point it is simple to drop Series that went over 7 games.

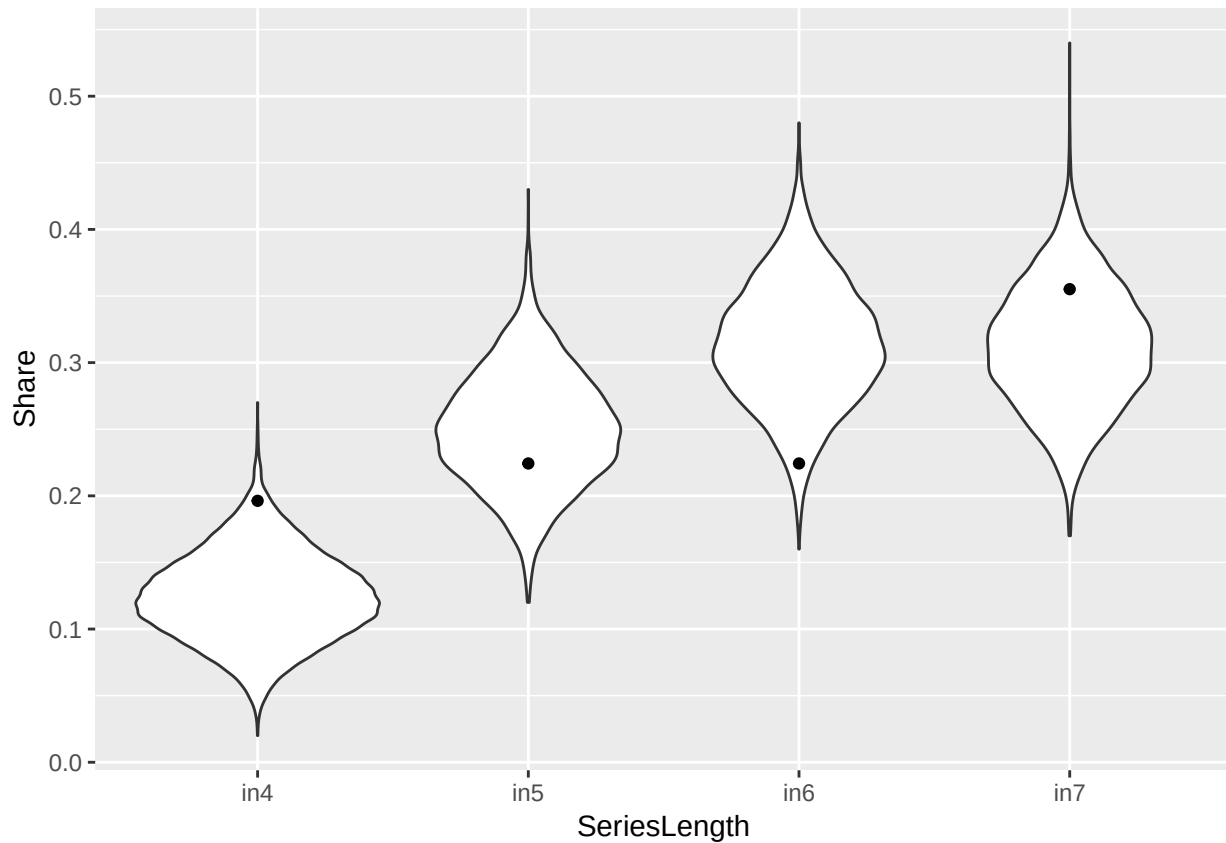
```
WSeriesin4 <- WSeriesin4 %>% filter(length < 8)
```

Given these data, it is now straightforward to plot the results:

```
eprops <- table(WSeriesin4$length)/length(WSeriesin4$length)
```

```
ActualWS <- data.frame(key = paste0("in", 4:7), value = as.numeric(eprops))
```

```
ggplot(topplot, aes(key, value)) + geom_violin() + labs(x = "SeriesLength", y = "Share") +
  geom_point(data = ActualWS)
```



As we can see, we simulated world series with as many or more 'done in 4' games 2.02 percent of the time. I would say we have evidence against the model. The problem is that once we have a low draw for one of the categories, we must have higher draws on at least one of the others. In the next section, I discuss the canonical way to test the fit between our data and this model.

Chi Square Test

Formally, we would use a chi-square test, a statistical procedure for testing whether overall distribution of events are generated by chance. As with other statistical tests, the goal is to see if the data is consistent with a null hypothesis. Our **null hypothesis** is that the World Series satisfies our model's theoretical proportions (0.125, 0.25, 0.3125, 0.3125).

If we have a theoretical model of counts and observed counts, we can test whether our data is consistent with our model using a Chi-square test. Specifically, the sum of squared deviations from the expected values are distributed by a Chi-square distribution. The procedure has us take the difference of the count of the observed categories from the expected counts, and square them. This is our squared difference. We normalize these differences by dividing by the expected count, and then sum across all our categories. This generates a single number, called a **test statistic**, that has a theoretical distribution.

$$\chi^2 = \sum_{i \in \text{Categories}} \frac{(\text{Observed}_i - \text{Expected}_i)^2}{\text{Expected}_i}$$

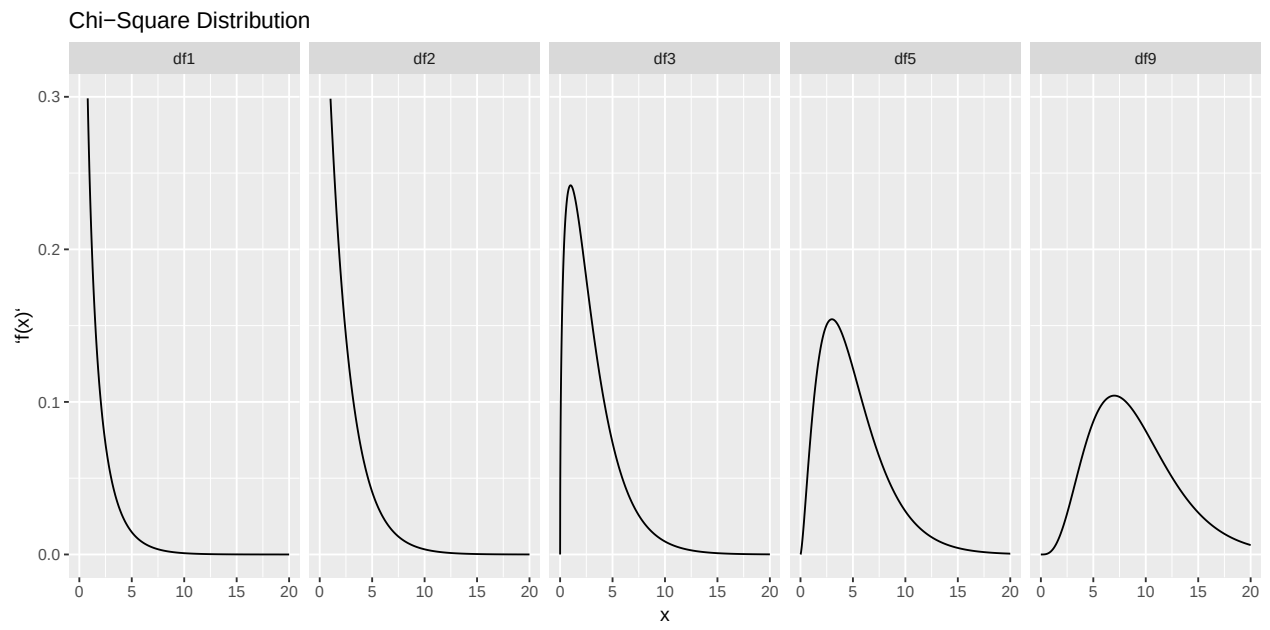
What we have done is taken a complicated model, our World Series model, and come up with a single statistical summary of the fit of that model to the data.

Chi-Square distributions have one parameter, the number of **Degrees of Freedom**. Normally, the **Degrees of Freedom** are the number of categories minus the number of fixed values in our theoretical model. In our example, we have four categories (done in 4, done in 5, etc.), and four expected proportions from our model. The four theoretical proportions must sum to one, so once we know three of the proportions, the fourth is set. This is an intuition for why we have three **Degrees of Freedom**.

The following code generates several Chi-Square distributions under different degrees of freedom. As you can see, the squared deviation is always expected to be positive, and depending on the number of degrees of freedom will have more mass at higher values of x. Because x will be the measure of deviation from our expected counts, you should intuitively think the more degrees of freedom, the more our test expects to see larger deviations just by chance.

```
degreesoffreedom <- c(1, 2, 3, 5, 9)
x <- seq(0, 20, by = 0.01)
chisquares <- map(degreesoffreedom, ~dchisq(x, df = .x))
names(chisquares) <- paste0("df", degreesoffreedom)

data.frame(chisquares) %>% mutate(x = x) %>% gather(key = "degreesoffreedom", value = "f(x)",
  ~x) %>% ggplot() + geom_line(aes(x, `f(x)`)) + facet_grid(. ~ degreesoffreedom) +
  ylim(c(0, 0.3)) + xlim(c(0, 20)) + labs(title = "Chi-Square Distribution")
```



We can apply these ideas to our observed and expected world series lengths.

```
Observed <- table(WSeriesin4$length)
Expected <- length(WSeriesin4$length) * tprops
chisqstat <- sum(((Observed - Expected)^2)/Expected)
```

Now, recall Chapter 2, we can calculate the p-value of a statistic by integrating the area of the distribution to the right of our test statistic. It corresponds to the probability, under the null hypothesis, of having seen a statistic as high as what we calculate or higher.

```
integrate(dchisq, df = 3, lower = chisqstat, upper = Inf)
```

```
## 0.04778241 with absolute error < 3.9e-05
```

It looks like we have a p-value of 0.047, so we can say that we can reject the hypothesis that our data was generated by our the model.

As before, R has a built in way of calculating the same thing:

```
testWS <- chisq.test(table(WSeriesin4$length), correct = F, p = tprops)
```

```
testWS
```

```
##
## Chi-squared test for given probabilities
##
## data:  table(WSeriesin4$length)
## X-squared = 7.9159, df = 3, p-value = 0.04778
```

Problems

Use R to simulate the following problems:

- (a) If three players are dealt five cards from a standard 52 card deck, how what is the probability that at least one person gets exactly two aces?
- (b) Suppose that n letters are placed randomly in n envelopes, what is the probability that $n - 1$ letters will be placed in the correct envelope?
- (c) Suppose that n letters are placed at random in n envelopes, if q_n is the probability that no letter is placed in the correct envelope, which of the following has the largest q_n ? $n = 10$, $n = 21$, $n = 53$, $n = 100$.
- (d) If the n letters are placed in n envelopes, what is the expected number of letters that go to the correct destination if $n = 10$, $n = 20$, $n = 50$?